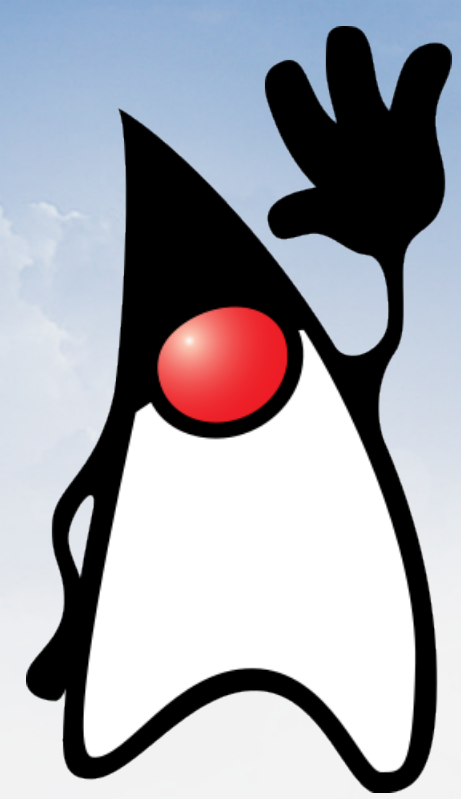




Java Basics2

COP 3330
Department of Computer Science
University of Central Florida
Dr. Bacanli

Variable naming rules

reserved words

abstract	assert	boolean	break	byte
case	catch	char	class	const
continue	default	do	double	else
enum	extends	final	finally	float
for	goto	if	implements	import
instanceof	int	interface	long	native
new	non-sealed	package	private	protected
public	return	short	strictfp	super
super	switch	synchronized	this	throw
throws	transient	try	void	volatile
while	_(underscore)			

Variable names cannot start with number.

`int 3number = 3; //`is invalid

We cannot use reserved keywords for variable names

`int goto = 5; //`invalid

`int Goto = 6; //` valid but not suggested

`int myfloat = 99; //` valid but not suggested

Variable names should not start with underscore `_` or dollar sign `$` characters, even though both are allowed.

Ethically:

The names of variables declared as constants should be all uppercase with words separated by underscores ("`_`").

`static final int GET_THE_CPU = 1;`

String Class

- Java has a class that manages Strings.
- A String is a sequence of Unicode characters.
- A library of methods exists for Strings
- Strings are not primitive data types like, ints, doubles, floats, chars, and booleans.

Concatenation

- Like most programming languages, we can combine strings.
- We can combine any strings and primitive types just by +
- Java makes it simple by using the + operator

```
String course = "COP";
```

```
String code = "333";
```

```
String coursecode = course + " " + code+0;
```

```
System.out.println(coursecode);
```

"COP 3330" would be displayed

Checking for Strings to be Equal

- To check if two strings are equal, we need to use `.equals`
- `==` won't work always!
- `==` should only be used on primitive types (ints, doubles, etc...)
- We will learn more later why `==` is not used for Strings and other class objects.
- For now, know that `equals()` method will check to see if strings are equal or not.
- There is also an `equals` method that is more flexible in checking by comparing upper and lowercase as the same. `equalsIgnoreCase()`
- For some cases, `==` may work with Strings but do not rely on comparing strings with `==`.

Length of Java Strings

- The String class has a built method for calculating the number of characters in a string.
- `length()` returns an integer, takes no parameters

```
package test;
public class testString{

    public static void main(String[] args){
        String someString=""; // What is the length of this?
        String name="John Doe";
        int lengthOfString = name.length();
        System.out.println( ( name ==name) + " " + (name == someString)); //true false
        System.out.println("length of name is "+lengthOfString);
        System.out.println(" are strings equal?" + name.equals(someString) );
    }
}
```

Conditions in Java

- Like many programming languages, Java has conditions
- Java also has switch statements.
- Nested if statements are allowed in Java
- Logical Operators
 - &&
 - ||
 - !
- Conditional Operator: expression1 ? expression2 : expresion3;
 - **int** y=8;
 - **int** x = y==9 ? 6 : 0;

if/else if/nested ifs

```
package leapTest;
public class LeapYearCheck {

    public static void main(String[] args) {
        int year = 2024; // always initialize variables inside the methods

        if ( (year % 4 == 0 && year % 100 != 0) || (year % 400 == 0) ) {
            System.out.println(year + " is a leap year.");
        } else { // make appropriate indentation for nested loops
            System.out.println(year + " is not a leap year."); // leave a space after punctuation marks
            if(year - 2000 > 0){
                System.out.println(year + " is later than 2000");
            }else if(year < 2000){
                System.out.println(year + " is before 2000");
            }else{
                System.out.println(year + " is 2000");
            }
            //end of else of year check 2000
        }
        // end of leap year checks  <- comments are useful for end of scopes, easy debugging
    }
}
```

switch

```
int day = 4;  
switch (day) {  
    case 6:  
        System.out.println("Today is Saturday");  
        break;  
    case 7:  
        System.out.println("Today is Sunday");  
        break;  
    default:  
        System.out.println("Looking forward to the Weekend");  
}  
// Outputs "Looking forward to the Weekend"
```

if no break; is done,
it will continue
execution.

DO NOT prefer switch. It is easy to make mistake by not adding break statement.
Use ifs instead.

parseInt, parseDouble

String to number conversions

```
String snumber = "45";
```

```
int number = Integer.parseInt(snumber);
```

```
double number2 = Double.parseDouble(snumber);
```

Number to String conversions

```
int number = 45;
```

```
String numberAsString=number + ""; //concat with empty string
```

parseInt in practice

```
package scannerTest;
import java.util.Scanner;

public class HelloWorld {
    public static void main(String[] args) {
        Scanner scan=new Scanner(System.in);
        String word=scan.next();    //gets everything until user presses Space

        System.out.println("You have entered " + word);

        int number = Integer.parseInt(word) * 2;
        System.out.println("Your number is half of " + number);

        scan.close();// close the input Stream
    } //end of main method
}
```

Loops in Java

- And just like with conditions, Java has loops!
- Loops allow us from rewriting the same set of consecutive statements repeatedly.
- for, while, and do-while loops like in C exists in Java
 - break statements (break out of the control structure of loop)
 - continue (skip an iteration)
- For a readable code, try not to use break and continue
 - You can add a boolean field in your code to do this
 - Go to considered harmful by Edsger W. Dijkstra

Loops

For and while loops are same in Java.

```
for(int i=0; i < limit; i++){  
    System.out.println(i);  
}
```

```
for(int i=0; i < limit; i++){  
    System.out.println(i);  
    i++;  
}
```

We can modify the index/counter variable inside the loop.

```
int i=0;  
while( i < limit ){  
    System.out.println(i);  
    i++;  
}
```

Loop scopes

```
for(int i=0; i < limit;i++){  
    System.out.println(i);  
    i++;  
}  
i=99;
```

The variable **i** is defined inside { }. It lives inside the { }. It will be destroyed after the scope. This statement will give an error.

```
int i=0;  
while(i < limit){  
    System.out.println(i);  
    i++;  
}  
i=99;
```

i is defined outside of the while loop. This statement will work.

do while

```
public static void main(String[] args){
    Scanner scan = new Scanner(System.in);
    String choice= ""; // null is possible, Always initialize! IMPORTANT
    do{
        System.out.println("Hi there");
        System.out.println("Write \"stop\" to stop greeting.");
        choice = input.nextLine();
    }while(!choice.equalsIgnoreCase("stop"));
} // end of main This comment will be very useful!
```

```
public static void main(String[] args){
    Scanner scan = new Scanner(System.in);
    String choice= "Yes";
    while(!choice.equalsIgnoreCase("stop")){
        System.out.println("Hi there");
        System.out.println("Write \"stop\" to stop greeting.");
        choice = scan.nextLine();
    } //end of while for choice input
}
```

do while does 1+
while does 0+

Variables inside the
functions/methods
should be always
initialized.

break/continue

```
int limit = 10;
for(int i=0; i < limit;i++){
    System.out.println(i);
    if( i == 8){
        break;
    }
}
//break goes here
```

break means leave the current loop.

```
int limit = 10;
for(int i=0; i < limit; i++){
    if( i >= 8){
        continue;
    }
    System.out.println(i);
    // continue goes here then up.
}
```

Continue means go to the end of the loop and do the next iteration.

break/continue

```
for(int i=0; i < limit;i++){  
    System.out.println(i);  
    if( i == 8){  
        break;  
    }  
}
```

```
boolean done = false;  
for(int i=0; i < limit && !done; i++){  
    System.out.println(i);  
    if( i == 8){  
        done = true;  
    }  
}
```

break and continue statements can be avoided using a boolean variable.

Characters ASCII and Unicode

- Java can be written in another language besides using English characters.
- You should of learned about ASCII (American Standard Code for Information Interchange)
 - This allows us to represent English characters using a system
- What about International Standards?
 - That is where Unicode comes in.
 - Unicode is a universal international standard character encoding
 - Unicode uses the hexadecimal system

```
public static void main(String[] args)
{
    char atsign=64;
    System.out.println(atsign);// @
    System.out.println((int)atsign); // 64
}
```

Typecasting

- Java allows primitive data types to be converted into a different primitive types.
- Implicit and Explicit casting
 - Implicit is automatic
 - Explicit is manual
- Examples
 - int → double (OK)
 - double → int (loss of information)
 - int → float (ok)
 - float → int (loss of information)

Typecasting examples

```
package somePackage;
public class testString{

    public static void main(String[] args) {
        int numberi=5;
        float numberf = 3.0f;

        numberf = numberi; // int can fit inside float
        System.out.println(numberi);
        System.out.println(numberf + "\n");

        numberf = 3.2f;
        numberi = (int)numberf; // float cannot fit inside int
        System.out.println(numberi);
        System.out.println(numberf);

    }
}
```

Java compiler warns you if you will lose information from castings.

You need to manually add some code to inform Java compiler that you are aware of this.

```
5
5.0

3
3.2
```

error: incompatible types:
possible lossy conversion from
float to int

Typecasting examples



```
package somePackage;
public class testString{

    public static void main(String[] args) {

        int numberi=5;
        float numberf = 3.0f;
        double numberd = 44.0;

        numberf = (float)numberd; // double is larger than float
        System.out.println("float number "+numberf);

        numberi = (int)numberd; // double cannot fit inside int
        System.out.println(numberi);
        System.out.println(numberf);

    }
}
```

Java compiler warns you if you will lose information from castings.

You need to manually add some code to inform Java compiler that you are aware of this.

```
float number 44.0
44
44.0
```

Data Type Chemistry

- You saw earlier about what happened when we combine ints and doubles in expressions.
- What happens here when Strings are involved?
`System.out.println(1 + 5.3 * 2 + "JAVA" + 2 + 6 * 4);`

